Caso 1 · Retail Andina

Caso 1 · Retail Andina

La empresa. Cadena de retail con 38 tiendas y un canal de venta en línea que ya pesa un tercio de la facturación. El negocio se mueve por campañas: Día de la Madre, Fiestas Patrias, Cyber y Navidad. En campaña se congela todo cambio que no sea corrección crítica. Fuera de campaña el equipo corre para dejar listo lo de la siguiente.

Cómo trabaja el equipo. Cinco equipos de desarrollo. Un monolito de catálogo y precios, más seis servicios que se le colgaron encima en los últimos tres años. Todo se integra contra el ERP, y el ERP solo existe en dos ambientes: producción y uno de pruebas. Ese ambiente de pruebas es único, y los cinco equipos se lo reparten por ventanas que se coordinan en una hoja compartida.

Lo que duele hoy.

- La ventana del ambiente es el recurso escaso del flujo. Un cambio pasa más tiempo esperando su ventana que siendo construido o probado.
- Los congelamientos de campaña acumulan cambios. Cuando se abre la ventana después del congelamiento, entra un lote grande y los defectos se mezclan.
- Los datos de prueba del ambiente se degradan. Cada equipo los deja como los necesita y el siguiente los encuentra distintos.

Validaciones de seguridad. El escaneo de seguridad lo corre el área de infraestructura contra la aplicación ya desplegada en ese ambiente. Ocupa una ventana completa. Esa ventana la pierde otro equipo, y el equipo de desarrollo recibe el resultado en un correo, días después, cuando ya está trabajando en otra cosa. El escaneo revisa de todo: dependencias, código y configuración del servidor, sin distinguir qué necesita la aplicación corriendo y qué no.

Insights:

- **Gestión de ventanas de**

Definan.

1. ¿Qué parte del escaneo actual necesita la ventana del ambiente? ¿Qué parte no la necesita y debería ser parte de las validaciones que corren en el pipeline?.
2. ¿Respecto a los controles que pasarían al pipeline, qué controles de seguridad detienen y cuáles sólo registran?
3. Tenemos un pase de emergencia en plena campaña, el nuevo gate de seguridad de pipeline detecta vulnerabilidad y no deja pasar. ¿Qué deberíamos hacer y por qué?
4. ¿Qué métrica DORA movemos con estos cambios?

Caso 2 · Financiera Los Andes

Caso 2 · Financiera Los Andes

La empresa. Financiera de créditos de consumo con colocación en tiendas aliadas y un app de clientes. El área de riesgos tiene peso en las decisiones de tecnología. Hay una auditoría interna anual y un comité de cambios a producción que sesiona una vez por semana.

Cómo trabaja el equipo. Cuatro equipos de desarrollo. Backend en Java, un par de servicios en .NET, el app en React Native. Release mensual planificada, además de pases de parches cuando algo se rompe. Cada release pasa por el comité antes de salir.

Lo que duele hoy.

- El gobierno está al final del flujo. Un cambio termina de construirse y entra a una cola de revisiones que se atiende por lotes, una vez por semana.
- La lista de verificación de seguridad vive en una hoja de cálculo. Se llena a mano, con el código ya congelado y la fecha ya comprometida con negocio.
- Un solo analista conoce la lista completa. Cuando está de vacaciones, la revisión se posterga o se aprueba sin revisar.
- La evidencia para la auditoría se arma a mano en carpetas compartidas, días antes de que llegue el auditor.

Validaciones de seguridad. El analista revisa el mismo conjunto de controles en cada entrega, leyendo. Una parte de esa lista es verificable de forma automática: versiones de dependencias contra vulnerabilidades conocidas, patrones de código ya catalogados, presencia de credenciales en el repositorio. La otra parte exige criterio y ninguna herramienta la cubre. Hoy las dos partes se revisan igual, con el mismo esfuerzo y en el mismo momento.

Definan.

1. ¿Qué controles de la lista se automatizan en el pipeline? ¿Cuáles se quedan con el analista y por qué?
2. Respecto a los controles que pasan al pipeline, ¿cuáles detienen y cuáles solo registran?
3. El pipeline aprobó un release y salió a producción. Tres semanas después, la auditoría interna encuentra una vulnerabilidad que la lista del analista sí habría detectado. ¿Qué pasó? ¿Es culpa del analista o de la herramienta?¿Qué cambian para la próxima?
4. ¿Qué métrica DORA movemos con estos cambios?

Caso 3 · Seguros Sur

CASO 3 · SEGUROS SUR

La empresa. Aseguradora con portal de cliente, app de siniestros y un core de pólizas que tiene doce años. El core cambia poco y con miedo. El portal y el app cambian seguido.

Cómo trabaja el equipo. Tres equipos de producto y un equipo de mantenimiento del core. El portal corre sobre un framework que va dos versiones mayores (major version) atrás de la actual. Nadie ha querido asumir la migración porque no hay una fecha de negocio que la justifique.

El pipeline hoy. El portal y el app tienen pipeline en GitHub Actions: instala dependencias, construye, corre pruebas unitarias y publica el artefacto. Corre en cada push a cualquier rama y toma seis minutos. No hay ninguna validación de seguridad. El core se construye y se despliega a mano.

Lo que duele hoy.

- El retrabajo es alto y no se ve. Una parte grande del trabajo en curso son correcciones a cosas ya entregadas, y el tablero las registra como historias nuevas.
- No hay forma de distinguir en el tablero qué es funcionalidad y qué es corrección. Cuando se mide velocidad, la corrección cuenta como avance.
- El framework desactualizado bloquea la actualización de varias librerías que dependen de él.

Validaciones de seguridad. El área de riesgos contrata una auditoría externa trimestral. El auditor entrega un informe con vulnerabilidades en dependencias. El equipo recibe una lista de hallazgos sobre código que liberó semanas atrás y que ya cambió. Se abren tickets, se priorizan contra el trabajo del trimestre, y una parte queda sin atender hasta el siguiente informe, donde vuelve a aparecer. Entre informe e informe, nadie revisa una dependencia nueva antes de que entre al proyecto.

Definan.

1. De los controles vistos hoy, ¿cuáles activan en este pipeline y cuáles descartan? ¿Por qué?
2. Para cada control que activan: ¿en qué punto corre (cada push, antes de fusionar, antes de publicar) y qué pasa cuando encuentra algo? ¿Detiene o solo registra?
3. Activan el análisis de composición. La primera ejecución devuelve más de 400 hallazgos, 60 de ellos críticos, y 35 de esos 60 solo se corrigen migrando el framework. La migración no tiene fecha ni presupuesto. ¿Con qué umbral configurarán el pipeline para que empiece a bloquear desde el lunes? ¿Por qué?
4. ¿Qué métrica DORA van a mover con estas mejoras?